

Security Evaluation of Projects (Negative Scoring System)

Objective

This document introduces a **standardized and quantitative approach** to evaluate the security posture of software projects. The methodology focuses on identifying vulnerabilities and weaknesses across key security domains, enabling teams to **detect, measure, and prioritize** security risks consistently.

#	Category (Multiple)	Title	Importance / Negative Score	Description / Additional Notes	Method / Tools	Further Reading Links
1	OWASP, Auth	A01: Broken Access Control – Secure Authentication and Precise Access Control	Critical (15 - 20)	Prevent unauthorized access by implementing proper AuthN and AuthZ (Includes MFA, login attempt limitation, and strict access control on API, UI, and Backend)	Use ASP.NET Identity, IdentityServer4, OAuth2, Policy-based Authorization, limit login attempts	OWASP Broken Access Control
2	OWASP, Cryptography	A02: Cryptographic Failures	Critical (15 - 20)	Weak cryptography leads to data exposure (Covers data storage and transmission)	Use strong, standard encryption algorithms (AES256, SHA512)	OWASP Cryptographic Failures
3	OWASP, Database, Injection	A03: Injection – Prevent SQL, Command, and XSS Injections	Highest (12 - 17)	Injection may expose or manipulate data, or execute malicious code (Includes SQL Injection, Command Injection, XSS)	Use parameterized queries with ORM (EF, Dapper), stored procedures, AntiXSS libraries	OWASP SQL Injection Cheat Sheet, OWASP Injection Prevention
4	OWASP	A04: Insecure Design	High (9 - 14)	Poor design allows security breaches (System architecture and design)	Design using Principle of Least Privilege (Threat Modeling)	OWASP Secure Design
5	OWASP, Security Misconfiguration	A05: Security Misconfiguration	High (9 - 14)	Misconfigured services/environments (Server, API, Frontend configurations)	Disable Debug, configure CORS properly (Security Scanners)	OWASP Security Misconfiguration
6	OWASP, Dependency Management	A06: Vulnerable & Outdated Components	High (9 - 14)	Vulnerable dependencies can compromise system security (Project files like package.json, csproj, pom.xml, requirements.txt)	Regular dependency scanning and updates, avoid unsupported packages (OWASP Dependency-Check, GitHub Dependabot, Snyk, Whitesource)	OWASP A06:2021
7	OWASP, Auth	A07: Identification & Authentication Failures	Critical (15 - 20)	Weak auth mechanisms (Authentication system)	Use MFA, limit login attempts (ASP.NET Identity, OAuth2)	OWASP Authentication Failures
8	OWASP	A08: Software and Data Integrity Failures	Medium (6 - 11)	Ensure integrity of software and data (CI/CD, input files)	Code signing, file validation (Code Signing Tools)	OWASP Data Integrity
9	Logging, Monitoring	A09: Security Logging and Monitoring	High (9 - 14)	Monitor events, prevent leakage of sensitive log data (Centralized logging, remove sensitive data, connect alarms)	Custom logger for filtering + monitoring tools (ELK, Seq, Sentry, Grafana)	OWASP Logging Best Practices
10	OWASP, Network Security	A10: SSRF (Server-Side Request Forgery)	Highest (12 - 17)	Prevent malicious outbound requests (Server-side HTTP requests)	IP safelist, request filtering (Firewall Rules)	OWASP SSRF Prevention
11	Browser & Headers, Network Security	HTTPS & HSTS Enforcement	Highest (12 - 17)	Prevent MITM attacks (Enforce HTTPS, enable HSTS)	Middleware for HTTPS redirection and HSTS (ASP.NET Core Middleware)	OWASP Transport Layer Protection
12	Logging	Contextual Logging and Tracing	High (9 - 14)	Enhance traceability and error diagnosis (Add Correlation ID, User ID, IP, Endpoint)	Middleware to generate and enrich logs with context	OWASP Logging

					(Serilog, ELK)	Context
13	Logging	Use of Custom Logger	Low (3 - 8)	Control over logged data (Logging system)	Filter sensitive data before logging (Serilog, Nlog)	OWASP Secure Logging
14	Sessions & Cookies, Auth	Secure Session Management (Timeout, Regeneration)	High (9 - 14)	Prevent session hijacking (Session and cookie management)	Set expiration, regenerate session ID (ASP.NET Core Session)	OWASP Session Management
15	OWASP, Auth	Secure Configuration Management	High (9 - 14)	Misconfigured settings can be exploited (Config files, runtime environment)	Use Secret Manager, encrypt sensitive settings (Azure Key Vault, HashiCorp Vault)	OWASP Security Misconfiguration
16	Auth	Secure Authentication	Critical (15 - 20)	Prevent impersonation and attacks (Authentication system)	Use secure JWT or cookies (ASP.NET Identity, JWT)	OWASP Auth
17	Financial & IPG, Auth	IP/Callback URL Validation	Highest (12 - 17)	Prevent fake/malicious callbacks (Payment & callback systems)	Whitelist allowed URLs/IPs (Firewall, IP Whitelist)	OWASP Payment Security
18	API Security	Rate Limiting / Throttling	High (9 - 14)	Prevent Brute-force and DoS attacks (APIs)	Middleware for rate limiting (AspNetCoreRateLimit)	OWASP Rate Limiting
19	Auth	Multi-Factor Authentication (MFA)	High (9 - 14)	Enhance authentication security (Admin and withdrawal sections)	Enable MFA (IdentityServer4, Authenticator Apps)	OWASP MFA
20	Auth, Network Security	Geo/IP Restrictions	Medium (6 - 11)	Restrict access to sensitive areas (Admin panel, transactions)	Limit by IP or country (Firewall Rules)	OWASP Network Security
21	Auth	Logout and Token Revocation	Medium (6 - 11)	Invalidate tokens and force re-login (Auth system)	Manage tokens and logout logic (ASP.NET Identity)	OWASP Token Management
22	Browser & Headers	Restrictive CORS Usage	High (9 - 14)	Prevent unauthorized API access (APIs)	Limit domains and methods (ASP.NET Core CORS)	OWASP CORS
23	OWASP, Validation, Auth	Prevent XSS, CSRF, and Validate/Sanitize Inputs	Critical (15 - 20)	Prevent script injection and forged requests (Forms, APIs)	AntiXSS, Encoding, Validation, AntiForgeryToken, CORS config (FluentValidation, DataAnnotations)	OWASP XSS, Input Validation, CSRF Prevention
24	Browser & Headers, Auth	Secure Cookies and Browser Headers	Highest (12 - 17)	Prevent cookie theft, XSS, CSRF, info leaks (Cookies and HTTP responses)	Set Secure, HttpOnly, SameSite, CSP, HSTS, etc. via Middleware (ASP.NET Core CookieOptions, Middleware)	OWASP Secure Headers

⚙️ Scoring Mechanism

- The system uses a **"negative score"** approach: the more security issues a project has, the **higher (worse)** the total score.
- Each **security category** (e.g., Authentication, Input Validation, Browser Headers, etc.) has a **defined negative score range** (e.g., 10 to 30 points).
- If a **confirmed vulnerability** is found in a category, the project receives a score **within that category's range**.
- If only **suspicious or incomplete code** is found (not a confirmed bug), a **lower-than-minimum score** may be applied (e.g., 5 or 8).
- Categories with **no issue** receive **zero** negative points.



This is a **penalty-based scoring system**, not a reward-based one:
A lower total score indicates better security.



Score Range

- The total negative score can range from **0 (highly secure)** to **380 (highly insecure)**.
- This reflects the cumulative score across all evaluated categories, assuming all categories contain vulnerabilities in the worst case.



Common Security Categories

Each project is evaluated across multiple security categories. These include but are not limited to:

Security Category	Description
Authentication & Sessions	Secure login, session fixation, JWT handling, refresh tokens
Input Validation	Protection against XSS, SQL Injection, Regex Injection
Output Encoding	HTML escaping, safe rendering to prevent reflected XSS
Authorization & Access Control	Role-based access, resource protection
Secure Headers	CSP, HSTS, X-Frame-Options, X-Content-Type-Options, etc.
TLS/HTTPS	SSL certificates, HTTPS enforcement, TLS settings
Logging & Auditing	Secure logging, no sensitive data leakage
Error Handling	Generic error messages, no internal structure exposure
Dependency Management	Vulnerabilities in packages (using Snyk, OWASP tools)
File Upload Security	MIME type restrictions, extension filtering, virus scanning
...	Other relevant categories based on the project stack



Use Cases in Development Lifecycle

This evaluation model can be applied at various stages of the software lifecycle:

Stage	Purpose
Early Development	Use as a secure coding checklist
Code Reviews	Assess newly introduced security risks
Pre-Production	Conduct security assessment before Go-Live
Post-Release Audits	Perform regular health checks on production systems

Security Level Classification (Based on Total Negative Score)

Security Level	Negative Score Range	Description
 Highly Secure	0 – 45	No known vulnerabilities, full and correct configurations, no critical issues
 Trustworthy	46 – 100	Minor issues or suspicious areas exist, but no confirmed bugs
 Needs Improvement	101 – 200	Multiple vulnerabilities found; can be exploited by attackers
 Critical	201+	Severe and widespread vulnerabilities; high risk of compromise

Additional Notes

- The scoring logic is based on **OWASP guidelines**, industry best practices, and internal security auditing experience.
- By scoring each project numerically, it becomes easier to **compare security health** across projects or over time.
- The model can serve as the foundation for a **security dashboard or health metric** across the organization.
- When tracked over multiple sprints or releases, this evaluation can reveal **trends in security improvements or regressions**.